

Função *lambda*

A **lambda** é uma função pequena, sem nome, escrita numa expressão só. Ela aparece principalmente **como argumento** de outras funções: dizer ao `sorted()` por qual critério ordenar, ao `max()` o que comparar, ao `filter()` o que manter. Nesta aula: a sintaxe, `key=`, `map`, `filter` e `reduce`, os limites da lambda e quando um `def` é melhor.

`lambda x: expressão`

`sorted(key=)`

`max · min`

`map · filter`

`reduce`

Limites

PEP 8

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Lambda Expressions; Sorting HOW TO; Built-in Functions: map, filter; functools.reduce) e PEP 8, consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Uma função *em uma expressão*

basico.py

```
def dobro(x):  
    return x * 2  
  
dobro_1 = lambda x: x * 2      # mesma coisa, em uma expressão  
print(dobro(5), dobro_1(5))  
soma = lambda a, b: a + b  
print(soma(2, 3))
```

SAÍDA

```
10 10  
5
```

LAMBDA
palavra-chave



x:
parâmetros



x * 2
o valor devolvido

- ✓ Não tem **return**: o resultado da expressão é devolvido
- ✓ Aceita vários parâmetros: **lambda a, b: a + b**
- ✓ Este exemplo dá nome à lambda só para comparar; na prática não se faz isso (seção 06)

Ordenando *por um critério*

sorted_key.py

```
alunos = [("Ana", 9.5), ("Bruno", 7.0), ("Carla", 8.5)]
por_nota = sorted(alunos, key=lambda a: a[1], reverse=True)
print(por_nota)
palavras = ["Python", "é", "muito", "legal"]
print(sorted(palavras, key=lambda p: len(p)))
print(max(alunos, key=lambda a: a[1]))
```

SAÍDA

```
[('Ana', 9.5), ('Carla', 8.5), ('Bruno', 7.0)]
['é', 'muito', 'legal', 'Python']
('Ana', 9.5)
```

- ✓ `key=` recebe uma função que diz o que comparar em cada item
- ✓ `lambda a: a[1]` : compare pela nota (posição 1 da tupla)
- ✓ `lambda p: len(p)` : compare pelo tamanho da palavra
- ✓ Funciona igual em `max()`, `min()` e `.sort()`

Esse é o uso nº 1 da lambda. Você já viu `key=str.lower` na aula 09: lá a função já existia com nome.

Ordenando *um dicionário*

dict_ordenar.py

```
estoque = {"maçã": 12, "banana": 30, "uva": 5}
mais = sorted(estoque.items(), key=lambda p: p[1], reverse=True)
print(mais)
print(sorted(estoque, key=lambda f: estoque[f]))  # só os nomes
```

SAÍDA

```
[('banana', 30), ('maçã', 12), ('uva', 5)]
['uva', 'maçã', 'banana']
```

- ✓ `.items()` dá pares (chave, valor); `p[1]` é o valor
- ✓ Resultado: lista de tuplas do maior estoque ao menor
- ✓ Ordenando só as chaves, a lambda consulta o dicionário:
`estoque[f]`

Transformar *e filtrar*

map_filter.py

```
numeros = [1, 2, 3, 4, 5, 6]
print(list(map(lambda n: n * 10, numeros)))
print(list(filter(lambda n: n % 2 == 0, numeros)))
# Com list comprehension (aula 16): costuma ser mais claro
print([n * 10 for n in numeros])
print([n for n in numeros if n % 2 == 0])
```

SAÍDA

```
[10, 20, 30, 40, 50, 60]
[2, 4, 6]
[10, 20, 30, 40, 50, 60]
[2, 4, 6]
```

- ✓ `map(função, coleção)` : aplica a função a cada item
- ✓ `filter(função, coleção)` : mantém os itens em que a função dá verdadeiro
- ✓ Os dois devolvem um iterador: `list()` mostra o resultado

Recomendação do guia (cap. 16): na maioria dos casos, prefira **list comprehension** (aula 16), que é mais "pythônica". As duas últimas linhas dão o mesmo resultado.

Acumulando *com reduce*

reduce_ex.py

```
from functools import reduce
precos = [10, 20, 30]
print(reduce(lambda total, p: total + p, precos))    # 60
print(sum(precos))    # mais simples
```

60

60

- ✓ `reduce` aplica a função de dois em dois, acumulando: $((10 + 20) + 30)$
- ✓ Fica no módulo `functools`
- ✓ Para somar, `sum()` é mais simples; para multiplicar, `math.prod()`

Vale conhecer para ler código de outras pessoas. No seu, prefira as funções prontas ou um `for` com acumulador (aula 14).

Quando usar *def*

limites.py

```
# lambda é uma única EXPRESSÃO: nada de blocos ou várias linhas  
faixa = lambda n: "alto" if n > 10 else "baixo" # ternário pode  
print(faixa(15), faixa(3))
```

```
# PEP 8: não dê nome a uma lambda; para isso existe def  
def faixa_def(n):  
    return "alto" if n > 10 else "baixo"  
print(faixa_def(15))
```

SAÍDA

```
alto baixo  
alto
```

- ✓ Uma lambda é **uma expressão**: sem várias linhas, sem `for`, sem bloco `if` (o ternário pode)
- ✓ Não tem docstring e aparece como `<lambda>` nas mensagens de erro
- ✓ A PEP 8 recomenda **não** dar nome a uma lambda (`f = lambda ...`): para isso existe `def`

Regra simples: lambda para uma função **pequena e descartável**, passada como argumento. Precisou de nome, explicação ou mais de uma linha: `def` .

Ordenando *produtos*

Enunciado: numa lista de produtos (dicionários), mostre os nomes do mais barato ao mais caro, só os que têm estoque e o produto com mais estoque.

Uma resposta possível

```
# Exercício: ordenar produtos de várias formas
produtos = [{"nome": "Café", "preco": 18.9, "estoque": 4},
            {"nome": "Arroz", "preco": 6.5, "estoque": 20},
            {"nome": "Azeite", "preco": 32.0, "estoque": 0}]
baratos = sorted(produtos, key=lambda p: p["preco"])
print([p["nome"] for p in baratos])
disponiveis = list(filter(lambda p: p["estoque"] > 0, produtos))
print([p["nome"] for p in disponiveis])
print(max(produtos, key=lambda p: p["estoque"])["nome"])
```

SAÍDA

```
['Arroz', 'Café', 'Azeite']
['Café', 'Arroz']
Arroz
```

- ✓ `key=lambda p: p["preco"]` : ordena pelo campo do dicionário
- ✓ Junta lambda, dicionários (aula 12) e list comprehension (aula 16)
- ✓ Desafio: ordene por estoque e, no empate, por nome (dica: a lambda pode devolver uma tupla)

Das coleções às funções

AULA	ASSUNTO	IDEIA CENTRAL
11 e 12	Set e dicionário	Valores únicos; pares chave: valor com .get()
13	Condições	if/elif/else, recuo, a ordem importa
14 e 15	for e while	Percorrer coleções; repetir até uma condição mudar
16	List comprehension	Transformar e filtrar numa linha, se continuar legível
17 e 18	Funções e argumentos	return, escopo, nomeados, padrão None
19	Recursão	Caso base + problema menor
20	*args e **kwargs	Recolher e espalhar argumentos
21	Lambda	Funções pequenas como argumento: key=

Com isso, você tem a base da linguagem. Os próximos passos estão nos capítulos 11 a 20 do **Guia Completo Python** (aula 01): arquivos, erros, módulos, classes e testes.

Teste o que *ficou*

1. `(lambda x, y: x * y)(3, 4)` vale:

A) 7

B) 12

C) 34

D) Erro

B.

2. Para ordenar tuplas (nome, nota) pela nota:

A) `sorted(lista)`

B) `sorted(lista, key=lambda t: t[1])`

C) `lista.sort(t[1])`

D) `sorted(lista[1])`

B.

3. O que uma lambda NÃO pode ter?

A) Parâmetros

B) Um ternário

C) Várias linhas e um bloco `for`

D) Um valor de retorno

C. É uma única expressão.

4. `list(filter(lambda n: n > 2, [1, 2, 3, 4]))` vale:

A) [1, 2]

B) [3, 4]

C) [True, True]

D) 2

B.

Funções pequenas, *na hora certa*

`lambda` parâmetros: `expressão` cria uma função sem nome, numa linha. Seu lugar é como argumento: `key=` em `sorted`, `max` e `min`, e às vezes em `map` e `filter` (quase sempre trocáveis por list comprehension). Precisou de nome ou de mais de uma linha, use `def`.

`lambda x: ...`

`key=`

`map · filter`

`reduce`

Nome? use `def`

Fim da aula 21 · Python Fundamentos

Fontes: docs.python.org/3/tutorial/controlflow.html#lambda-expressions; docs.python.org/3/howto/sorting.html; docs.python.org/3/library/functions.html (`map`, `filter`, `sorted`, `max`); docs.python.org/3/library/functools.html#functools.reduce; peps.python.org/pep-0008, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.